ShanghaiTech University

CS240 Algorithm Design and Analysis

Spring 2026

Problem Set 3

Due: 23:59, March 26, 2026

1. Please write your solutions in English.
2. Submit your solutions to **Gradescope**. When submitting, match your solutions to the problems correctly.
3. If you want to submit a handwritten version, scan it clearly.
4. Up to 2 late days are allowed for homework submissions. Assignments submitted within the first 24 hours after the deadline will incur a 10% penalty; those submitted within 48 hours will incur a 20% penalty. Submissions more than 48 hours late will not be accepted unless there is a valid reason, such as a medical or family emergency.
5. You are required to follow ShanghaiTech's academic honesty policies. You are allowed to discuss problems with other students, but you must write up your solutions by yourselves. You are not allowed to copy materials from other students or from online or published resources. Violating academic honesty can result in serious penalties.

Notice: When proving that a problem A is NP-complete, you need to strictly follow the steps below and explicitly state each part.

- Show that $A \in \text{NP}$.

  That is, given a candidate solution, we can verify it in polynomial time.

- Choose a known NP-complete problem B.

- Give a polynomial-time reduction from B to A.

  For any instance b of B, construct an instance a of A in polynomial time.

- Prove correctness of the reduction.

  Show that

  $$b \text{ is a YES-instance of } B \quad \Longleftrightarrow \quad a \text{ is a YES-instance of } A.$$

Problem 1:

Given:

- A set of communication links

$$L = \{\ell_1, \ell_2, \ldots, \ell_n\}.$$

- A set of interfering pairs

$$I = \{(\ell_{a_1}, \ell_{b_1}), (\ell_{a_2}, \ell_{b_2}), \ldots, (\ell_{a_m}, \ell_{b_m})\} \subseteq L \times L,$$

  where each pair $(\ell_i, \ell_j) \in I$ means that links ℓ_i and ℓ_j cannot be activated simultaneously.

- An integer k.

Question:

Does there exist a subset $L' \subseteq L$ such that:

1. $|L'| \geq k$, and
2. for every interfering pair $(\ell_i, \ell_j) \in I$, at most one of ℓ_i or ℓ_j belongs to L'?

Prove that this problem is NP-complete.

Solution:

1. Proof that the problem is in NP

Given a candidate solution $L' \subseteq L$, we can verify in polynomial time whether it satisfies the two required conditions:

- Check whether $|L'| \geq k$.
- Check whether for every interfering pair $(\ell_i, \ell_j) \in I$, at most one of ℓ_i and ℓ_j belongs to L'.

To verify a candidate solution $L' \subseteq L$, we first create a Boolean array of length $|L|$ indicating whether each link is selected in L', which takes $O(|L|)$ time. Then, we scan all interfering pairs in I once; for each pair (ℓ_i, ℓ_j), we check whether both endpoints are selected, which takes $O(|I|)$ time in total. Therefore, the total verification time is

$$O(|L| + |I|),$$

which is polynomial in the input size. Hence, the problem belongs to NP.

2. Polynomial-Time Reduction from Independent Set

We reduce from Independent Set, which is known to be NP-complete.

Consider an arbitrary instance of Independent Set. We are given an undirected graph

$$G = (V, E)$$

and an integer k. A subset of vertices $S \subseteq V$ is called an independent set if no two vertices in S are adjacent. The question is whether there exists an independent set of size at least k.

From this instance, we construct an instance of the link activation problem in polynomial time. For each vertex $v_i \in V$, create a communication link ℓ_i, and let

$$L = \{\ell_i \mid v_i \in V\}.$$

For each edge $(v_i, v_j) \in E$, add the pair (ℓ_i, ℓ_j) to the interference set, and define

$$I = \{(\ell_i, \ell_j) \mid (v_i, v_j) \in E\}.$$

The resulting instance of our problem is then

$$(L, I, k),$$

which asks whether there exists a subset $L' \subseteq L$ with $|L'| \geq k$ such that no two links in L' interfere.

This construction takes $O(|V|+|E|)$ time, since we create exactly one link for each vertex in V and exactly one interfering pair for each edge in E. Therefore, the reduction is polynomial-time.

3. Correctness of the Reduction

We now prove that G has an independent set of size at least k if and only if the constructed link activation instance (L, I, k) is feasible.

($\Rightarrow$) Suppose that G has an independent set $S \subseteq V$ with $|S| \geq k$. By definition, no two vertices in S are adjacent in G, so for any two vertices $v_i, v_j \in S$, we have $(v_i, v_j) \notin E$. By construction, a pair (ℓ_i, ℓ_j) belongs to I if and only if $(v_i, v_j) \in E$. Hence, among the links corresponding to the vertices in S, no interfering pair appears. Let

$$L' = \{\ell_i \mid v_i \in S\}.$$

Then $|L'| = |S| \geq k$, and no two links in L' form an interfering pair. Therefore, L' is a feasible solution to the constructed instance, so (L, I, k) is feasible.

($\Leftarrow$) Suppose that (L, I, k) is feasible. Then there exists a subset $L' \subseteq L$ such that $|L'| \geq k$ and no two links in L' form an interfering pair. Define

$$S = \{v_i \mid \ell_i \in L'\}.$$

Then $|S| = |L'| \geq k$. We claim that S is an independent set in G. Indeed, if there were an edge $(v_i, v_j) \in E$ with both $v_i, v_j \in S$, then by construction $(\ell_i, \ell_j) \in I$. Since both ℓ_i and ℓ_j belong to L', this would contradict the fact that no two links in L' form an interfering pair. Therefore, no two vertices in S are adjacent, so S is an independent set of size at least k.

Therefore, G has an independent set of size at least k if and only if the constructed link activation instance (L, I, k) is feasible.

Since the link activation problem is in NP and Independent Set can be reduced to it in polynomial time, we conclude that this problem is NP-complete.

Problem 2:

ShanghaiTech is planning a night security patrol system on campus. The campus is modeled as an undirected graph $G = (V, E)$, where each vertex represents an area, and an edge between two vertices means that the two areas are directly connected.

If a patrol point is placed in an area, then that patrol point can monitor both the area itself and all areas directly connected to it.

The university would like to place at most k patrol points so that every area on campus is monitored.

Formally, given an undirected graph $G = (V, E)$ and a positive integer k, decide whether there exists a subset $S \subseteq V$ such that

- $|S| \leq k$, and
- for every vertex $v \in V$, either $v \in S$, or there exists some $u \in S$ such that $(u, v) \in E$.

Prove that this decision problem is NP-complete.

Solution:

1. Proof that Campus Patrol Point Placement is in NP

Given a candidate solution, namely a patrol-point set $S \subseteq V$, we verify whether it satisfies the required conditions.

First, we check whether $|S| \leq k$, which takes $O(|V|)$ time.

Then, for each vertex $v \in V$, we check whether $v \in S$, or whether v is adjacent to some vertex in S. If the graph is stored using adjacency lists, this process scans all vertices and all adjacency edges once, so the total running time is $O(|V| + |E|)$.

Therefore, the entire verification procedure is polynomial in the input size. Hence, Campus Patrol Point Placement is in NP.

2. Polynomial-Time Reduction from Vertex Cover

We reduce from Vertex Cover, which is known to be NP-complete.

Let (G, k) be an arbitrary instance of Vertex Cover, where

$$G = (V, E).$$

The question is whether there exists a subset $C \subseteq V$ with $|C| \leq k$ such that every edge in E has at least one endpoint in C.

We construct an instance (G', k) of Campus Patrol Point Placement as follows.

For each edge $e = (u, v) \in E$, introduce a new vertex x_e, and connect x_e to both u and v. No other new edges are added.

Thus, the new graph G' is obtained by:

- keeping all original vertices in V,
- adding one new vertex x_e for each edge $e \in E$,
- adding two new edges (x_e, u) and (x_e, v) for each edge $e = (u, v) \in E$.

The parameter k remains unchanged.

Since the original graph has $|V|$ vertic es and $|E|$ edges, the construction adds exactly $|E|$ new vertices and $2|E|$ new edges. Hence, the size of G' is $O(|V| + |E|)$, and the construction itself takes

$$O(|V| + |E|),$$

which is polynomial time.

3. Correctness of the Reduction

We now prove that (G, k) is a yes-instance of Vertex Cover if and only if (G', k) is a yes-instance of Campus Patrol Point Placement.

($\Rightarrow$) Suppose that (G, k) is a yes-instance of Vertex Cover. Then there exists a set $C \subseteq V$ such that $|C| \leq k$ and every edge in E has at least one endpoint in C.

We claim that C is a feasible patrol-point set in G'.

First, $|C| \leq k$ holds immediately.

Next, we show that every vertex of G' is monitored by C.

- Every vertex in C is clearly monitored.
- For every new vertex x_e, where $e = (u, v) \in E$, since C is a vertex cover of G, at least one of u and v belongs to C. Because x_e is adjacent to both u and v, it follows that x_e is adjacent to a vertex in C, and hence is monitored.

Therefore, every vertex in G' is either in C or adjacent to a vertex in C. So C is a valid solution to the constructed Campus Patrol Point Placement instance.

($\Leftarrow$) Suppose that (G', k) is a yes-instance of Campus Patrol Point Placement. Then there exists a set S of vertices in G' such that $|S| \leq k$ and every vertex in G' is monitored by S.

We construct a set $C \subseteq V$ as follows.

- If a vertex in S is an original vertex in V, keep it in C.
- If a vertex in S is a new vertex x_e, where $e = (u, v)$, replace x_e by either u or v.

In this way, C contains only original vertices, and its size does not increase. Hence,

$$|C| \leq |S| \leq k.$$

We now show that C is a vertex cover of G.

Take any edge $e = (u, v) \in E$. In the graph G', there is a corresponding new vertex x_e, and x_e is adjacent only to u and v.

Since S is a dominating set of G', the vertex x_e must be monitored. Therefore, one of the following must happen:

- $x_e \in S$, or
- at least one of u and v belongs to S.

If $x_e \in S$, then by construction we replace x_e by either u or v, so at least one endpoint of e belongs to C.

If at least one of u and v belongs to S, then that vertex is kept in C, so again at least one endpoint of e belongs to C.

Thus, every edge $e = (u, v) \in E$ has at least one endpoint in C. Therefore, C is a vertex cover of G with $|C| \leq k$.

Therefore, (G, k) is a yes-instance of Vertex Cover if and only if (G', k) is a yes-instance of Campus Patrol Point Placement.

Since Campus Patrol Point Placement is in NP and Vertex Cover can be reduced to it in polynomial time, we conclude that Campus Patrol Point Placement is NP-complete.

Problem 3:

Monotone 3-SAT: A *3-SAT formula* is a Boolean formula in which each clause contains exactly three literals. A clause is called *monotone* if all of its literals are positive or all of its literals are negative. For example, $(x \vee y \vee z)$ and $(\neg x \vee \neg y \vee \neg z)$ are monotone clauses, while $(x \vee y \vee \neg z)$ is not. As usual, a clause is satisfied if at least one of its literals evaluates to true.

Given a 3-SAT formula ϕ in which every clause of ϕ is monotone, decide whether ϕ has a satisfying assignment. Prove that MONOTONE 3-SAT is NP-complete.

Solution:

1. Proof that Monotone 3-SAT is in NP

Given a candidate truth assignment, we can verify in polynomial time whether it satisfies the given MONOTONE 3-SAT instance.

Specifically, we examine each clause one by one. Since the formula is in 3CNF, every clause contains exactly three literals. For each clause, we simply check whether at least one of its three literals evaluates to true under the given assignment. This takes constant time per clause. Therefore, if the formula contains m clauses, the total verification time is $O(m)$, which is polynomial in the input size.

Hence, MONOTONE 3-SAT is in NP.

2. Polynomial-Time Reduction from 3-SAT

We reduce from 3-SAT, which is known to be NP-complete.

Let

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

be an arbitrary instance of 3-SAT, where each clause contains exactly three literals.

We construct a formula ϕ' as follows. For each clause C_i, we consider whether it is already monotone.

Case 1: C_i is already monotone.

If all three literals in C_i are positive, or all three literals in C_i are negative, then we keep C_i unchanged.

Case 2: C_i contains both positive and negative literals.

Since C_i has exactly three literals, it must be of one of the following two forms:

$$(x \vee y \vee \neg z) \qquad \text{or} \qquad (x \vee \neg y \vee \neg z).$$

Case 2a: $C_i = (x \vee y \vee \neg z)$**.**

Introduce a fresh new variable u_i, and replace C_i by the two clauses

$$(x \vee y \vee u_i) \qquad \text{and} \qquad (\neg z \vee \neg u_i \vee \neg u_i).$$

The first clause is monotone positive, and the second clause is monotone negative. Each of them contains exactly three literals.

Case 2b: $C_i = (x \vee \neg y \vee \neg z)$**.**

Introduce a fresh new variable u_i, and replace C_i by the two clauses

$$(x \vee u_i \vee u_i) \qquad \text{and} \qquad (\neg y \vee \neg z \vee \neg u_i).$$

Again, the first clause is monotone positive, the second clause is monotone negative, and both clauses contain exactly three literals.

Applying this replacement independently to every clause of ϕ, we obtain a formula ϕ' in which every clause is monotone and every clause contains exactly three literals. The construction clearly takes polynomial time, since each original clause is replaced by at most two new clauses and introduces at most one fresh variable.

3. Correctness of the Reduction

We now prove that ϕ is satisfiable if and only if ϕ' is satisfiable.

(⇒) Suppose that ϕ is satisfiable. Let α be a satisfying assignment of ϕ. We show how to extend α to the new variables introduced in the construction so that ϕ' is also satisfied.

First, consider any clause C_i that was already monotone. Since it remains unchanged in ϕ', and since α satisfies every clause of ϕ, C_i is also satisfied in ϕ'. Now consider a clause of the form

$$C_i = (x \vee y \vee \neg z).$$

This clause is replaced by

$$(x \vee y \vee u_i) \qquad \text{and} \qquad (\neg z \vee \neg u_i \vee \neg u_i).$$

Since α satisfies C_i, at least one of x, y, and $\neg z$ is true under α.

If $x \vee y$ is true under α, then set $u_i = \text{false}$. In this case, the first new clause

$$(x \vee y \vee u_i)$$

is true because $x \vee y$ is already true, and the second new clause

$$(\neg z \vee \neg u_i \vee \neg u_i)$$

is also true because $\neg u_i$ is true.

If $x \vee y$ is false under α, then since C_i is satisfied, $\neg z$ must be true. In this case, set $u_i = \text{true}$. Then the first new clause is true because u_i is true, and the second new clause is true because $\neg z$ is true.Hence both new clauses are satisfied.

Next, consider a clause of the form

$$C_i = (x \vee \neg y \vee \neg z).$$

This clause is replaced by

$$(x \vee u_i \vee u_i) \qquad \text{and} \qquad (\neg y \vee \neg z \vee \neg u_i).$$

Since α satisfies C_i, at least one of x, $\neg y$, and $\neg z$ is true under α.

If x is true, set $u_i = \text{false}$. Then the first new clause is true because x is true, and the second new clause is true because $\neg u_i$ is true.

If x is false, then since C_i is satisfied, at least one of $\neg y$ and $\neg z$ must be true. In this case, set $u_i = \text{true}$. Then the first new clause is true because u_i is true, and the second new clause is true because $\neg y \vee \neg z$ is true. Thus both new clauses are satisfied.

Since this can be done for every clause, α can be extended to a satisfying assignment of ϕ'. Therefore, ϕ' is satisfiable.

($\Leftarrow$) Suppose that ϕ' is satisfiable. Let β be a satisfying assignment of ϕ'. We restrict β to the original variables of ϕ and show that this restricted assignment satisfies ϕ.

First, any clause of ϕ that was already monotone remains unchanged in ϕ'. Since β satisfies ϕ', it must also satisfy that original clause. Now consider an original clause

$$C_i = (x \vee y \vee \neg z),$$

which was replaced by

$$(x \vee y \vee u_i) \qquad \text{and} \qquad (\neg z \vee \neg u_i \vee \neg u_i).$$

Suppose, for contradiction, that the original clause C_i is false under the restricted assignment. Then

$$x = \text{false}, \qquad y = \text{false}, \qquad \neg z = \text{false}.$$

Hence the first new clause becomes

$$u_i,$$

so we must have $u_i = \text{true}$. But then the second new clause becomes

$$\neg u_i \vee \neg u_i,$$

which forces $u_i = \text{false}$. This is a contradiction. Therefore, the original clause C_i must be true.

Similarly, consider an original clause

$$C_i = (x \vee \neg y \vee \neg z),$$

which was replaced by

$$(x \vee u_i \vee u_i) \qquad \text{and} \qquad (\neg y \vee \neg z \vee \neg u_i).$$

Suppose, again for contradiction, that the original clause C_i is false under the restricted assignment. Then

$$x = \text{false}, \qquad \neg y = \text{false}, \qquad \neg z = \text{false}.$$

So the first new clause becomes

$$u_i \vee u_i,$$

which forces $u_i = \text{true}$. But then the second new clause becomes

$$\neg u_i,$$

which forces $u_i = \text{false}$. Again we get a contradiction. Therefore, the original clause C_i must be true. Hence every original clause of ϕ is satisfied under the restriction of β, so ϕ is satisfiable.

Therefore, ϕ is satisfiable if and only if ϕ' is satisfiable.

Since MONOTONE 3-SAT is in NP and 3-SAT can be reduced to it in polynomial time, we conclude that MONOTONE 3-SAT is NP-complete.

Problem 4:

Given a finite set of items I, where each item $i \in I$ has a positive integer size $s(i) \in \mathbb{Z}^+$, and two integer bin capacities B_1 and B_2. Is there a way to partition the set I into two disjoint subsets I_1 and I_2 such that every item belongs to exactly one subset, and

$$\sum_{i \in I_1} s(i) \leq B_1 \qquad \text{and} \qquad \sum_{i \in I_2} s(i) \leq B_2?$$

Prove this problem is NP-complete.
Hint: You may use a reduction from the NP-complete problem Subset Sum.

Solution:

1. Proof that the problem is in NP

Given a candidate solution I_1, I_2, we check whether I_1 and I_2 form a partition of I, and whether $\sum_{i \in I_1} s(i) \leq B_1$ and $\sum_{i \in I_2} s(i) \leq B_2$. This verification can be done by scanning all items and computing the two sums, which takes $O(|I|)$ time. Therefore, the verification is polynomial in the input size, and the problem is in NP.

2. Polynomial-Time Reduction from Subset Sum

We reduce from Subset Sum, which is NP-complete. In Subset Sum, given positive integers $a_1, a_2, \ldots, a_n$ and a target integer T, the question is whether there exists a subset whose sum is exactly T.

Given an instance $(a_1, a_2, \ldots, a_n, T)$ of Subset Sum, we construct an instance of our problem as follows. For each integer a_j, create an item $i_j \in I$ with size $s(i_j) = a_j$. Thus, the item set is $I = \{i_1, i_2, \ldots, i_n\}$. Let

$$A = \sum_{j=1}^{n} a_j$$

be the total size of all items. We set the two bin capacities to be $B_1 = T$ and $B_2 = A - T$. Therefore, the constructed instance asks whether the items in I can be partitioned into two disjoint subsets such that the first subset has total size at most T and the second subset has total size at most $A - T$. This construction takes polynomial time.

3. Correctness of the Reduction

We prove that the Subset Sum instance has a solution if and only if the constructed instance of our problem has a solution.

($\Rightarrow$) Suppose the SUBSET SUM instance has a solution. Then there exists a subset $S \subseteq \{1, 2, \ldots, n\}$ such that

$$\sum_{j \in S} a_j = T.$$

In the constructed instance, put the corresponding items into the first subset:

$$I_1 = \{i_j \mid j \in S\}.$$

Then

$$\sum_{i \in I_1} s(i) = \sum_{j \in S} a_j = T = B_1.$$

Put all remaining items into the second subset $I_2 = I \setminus I_1$. Since the total size of all items is A, we have

$$\sum_{i \in I_2} s(i) = A - T = B_2.$$

Therefore, both capacity constraints are satisfied, and the constructed instance has a valid solution.

($\Leftarrow$) Suppose the constructed instance of this problem has a solution. Then I can be partitioned into two disjoint subsets I_1 and I_2 such that

$$\sum_{i \in I_1} s(i) \leq B_1 = T \qquad \text{and} \qquad \sum_{i \in I_2} s(i) \leq B_2 = A - T.$$

Since I_1 and I_2 form a partition of I, we have

$$\sum_{i \in I_1} s(i) + \sum_{i \in I_2} s(i) = A.$$

On the other hand, by the two capacity constraints,

$$\sum_{i \in I_1} s(i) + \sum_{i \in I_2} s(i) \leq T + (A - T) = A.$$

Hence both capacity constraints must be tight. In particular,

$$\sum_{i \in I_1} s(i) = T.$$

Therefore, the integers corresponding to the items in I_1 form a subset whose sum is exactly T. Thus, the original SUBSET SUM instance has a solution.

Since the problem is in NP, and since we have given a polynomial-time reduction from the NP-complete problem SUBSET SUM and proved the equivalence in both directions, we conclude that this problem is NP-complete.

Problem 5:

Consider an $m \times n$ grid. Each cell is either empty, contains a black piece, or contains a white piece. You are allowed to remove some of the pieces. The question is whether it is possible to obtain a configuration such that

- each row contains at least one piece, and
- no column contains both black and white pieces.

Prove that this problem is NP-complete.

Solution:

1. Proof that the problem is in NP

Given a candidate solution, we verify whether it is valid in polynomial time as follows. First, for each row, we check whether at least one piece remains in that row. This can be done by scanning all entries of the grid, which takes $O(mn)$ time. Second, for each column, we check whether all remaining pieces in that column are of the same color, namely, either all black or all white, but not both. This can also be done by scanning the grid, and thus takes $O(mn)$ time. Therefore, the total verification can be completed in polynomial time. Hence, the problem is in NP.

2. Polynomial-Time Reduction from 3SAT

We reduce from 3SAT, which is NP-complete. Let Φ be a 3CNF formula with clauses $C_1, C_2, \ldots, C_m$ and variables $x_1, x_2, \ldots, x_n$. We construct an instance of our problem as follows.

We create an $m \times n$ grid, where each row corresponds to a clause and each column corresponds to a variable. For each clause C_i and variable x_j, we define the cell (i, j) as follows: if x_j appears in C_i, place a black piece in cell (i, j); if $\overline{x_j}$ appears in C_i, place a white piece in cell (i, j); otherwise, leave cell (i, j) empty.

This construction can clearly be carried out in polynomial time.

3. Correctness of the Reduction

We prove that the 3SAT instance Φ is satisfiable if and only if the constructed grid instance has a valid solution.

($\Rightarrow$) Suppose that Φ is satisfiable. Then there exists a truth assignment that satisfies every clause. For each variable x_j, consider column j of the grid. If

x_j = TRUE, remove all white pieces from column j. If x_j = FALSE, remove all black pieces from column j.

After this removal, each column contains pieces of only one color, so the column condition is satisfied. Moreover, since every clause contains at least one true literal under the satisfying assignment, each row still contains at least one remaining piece. Therefore, the constructed grid instance has a valid solution.

($\Leftarrow$) Suppose that the constructed grid instance has a valid solution. Then in every column, the remaining pieces are of at most one color. We define a truth assignment as follows. For each variable x_j, if column j contains at least one black piece, set x_j = TRUE; if column j contains at least one white piece, set x_j = FALSE; if column j is empty, assign x_j arbitrarily.

This assignment is well defined because no column contains both black and white pieces. Since each row contains at least one remaining piece, every clause has at least one literal whose corresponding piece remains in that row. If the remaining piece in row i and column j is black, then x_j appears positively in C_i and is assigned TRUE; if it is white, then $\overline{x_j}$ appears in C_i and x_j is assigned FALSE. In either case, clause C_i is satisfied. Hence every clause is satisfied, and therefore Φ is satisfiable.

Since the problem is in NP and is NP-hard, it is NP-complete.